D ×

Flask CRUD Application with SQLAlchemy

by Parwiz

In this article we are going to learn **Flask CRUD Application with SQLAlchemy**, we are going to build web application that you can create employee, read employee data from mysql database, update employee data and also delete employee. also we are using

SQLAlchemy for this tutorial, if your interested in the video for this course you can check below.

Python Web Development Tutorials

1: Flask Web Development Tutorials

2: Django Web Development Tutorials

Also you can read more articles on Python GUI Development

1: TKinter GUI Development Tutorials

2: Pyside2 GUI Development Tutorials

3: wxPython GUI Development Tutorials

4: Kivy GUI Development Tutorials

What is Flask ?

Flask is a web framework. This means flask provides you with tools, libraries and technologies that allow you to build a web application. This web application can be some web pages, a blog, a wiki or go as big as a web-based calendar application or a commercial website.

Flask is part of the categories of the micro-framework. Micro-framework are normally framework with little to no dependencies to external libraries. This has pros and cons. Pros would be that the framework is light, there are little dependency to update and watch for security bugs, cons is that some time you will have to do more work by yourself or increase yourself the list of dependencies by adding plugins.

What is SQLAlchemy ?

SQLAlchemy is the Python SQL toolkit and Object Relational Mapper that gives application developers the full power and flexibility of SQL.

It provides a full suite of well known enterprise-level persistence patterns, designed for efficient and high-performing database access, adapted into a simple and Pythonic domain language.

What is Flask-SQLAlchemy ?

Flask-SQLAlchemy is an extension for Flask that adds support for SQLAlchemy to your application. It aims to simplify using SQLAlchemy with Flask by providing useful defaults and extra helpers that make it easier to accomplish common tasks.

See the SQLAlchemy documentation to learn how to work with the ORM in depth. The following documentation is a brief overview of the most common tasks, as well as the features specific to Flask-SQLAlchemy.

Also you can watch the complete video for Flask CRUD Application with SQLAlchemy

Flask CRUD Application Full Course With SQLAlchemy | Python Flask

Installation

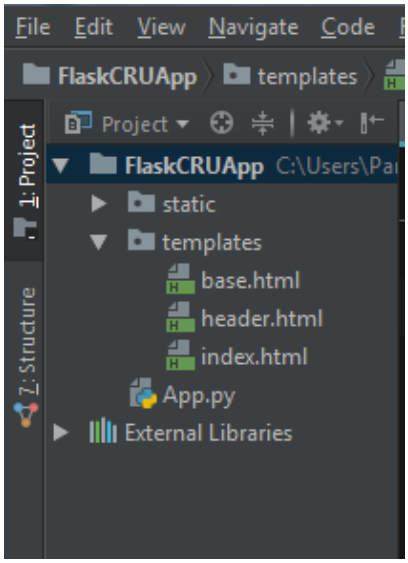
For this article first you need to install flask, and after that you need to install flask-sqlalchemy.

```
pip install Flask
```

```
pip install Flask-SQLAlchemy
```

OK after the installation first you need to create a database, iam using Wamp Server for this tutorial and my database name is crud. after that you need to open your Pycharm IDE and create two folders one for your static files and the second for your templates, in the video i have used bootstrap and jquery static files, but in here i have changed that to BootstrapCDN. in your templates you need to create some html files.

×



Flask CRUD Project Structure

So you can see i have static folder, but because iam using BootstrapCDN, iam not going to add bootstrap and jquery files in their. in the templates folder i have the HTML files that i need for this tutorial.

Let's add our codes in our *app.py*.

app.py Python

```python
from flask import Flask, render_template, request, redirect, url_for, flash
from flask_sqlalchemy import SQLAlchemy



app = Flask(__name__)
app.secret_key = "Secret Key"
```

```python

#SqlAlchemy Database Configuration With Mysql
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql://root:''@localhost/crud'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)


#Creating model table for our CRUD database
class Data(db.Model):
    id = db.Column(db.Integer, primary_key = True)
    name = db.Column(db.String(100))
    email = db.Column(db.String(100))
    phone = db.Column(db.String(100))


    def __init__(self, name, email, phone):

        self.name = name
        self.email = email
        self.phone = phone




#This is the index route where we are going to
#query on all our employee data
@app.route('/')
def Index():
    all_data = Data.query.all()

    return render_template("index.html", employees = all_data)



#this route is for inserting data to mysql database via html forms
@app.route('/insert', methods = ['POST'])
def insert():

    if request.method == 'POST':

        name = request.form['name']
        email = request.form['email']
        phone = request.form['phone']


        my_data = Data(name, email, phone)
        db.session.add(my_data)
        db.session.commit()

        flash("Employee Inserted Successfully")

        return redirect(url_for('Index'))


#this is our update route where we are going to update our employee
@app.route('/update', methods = ['GET', 'POST'])
def update():

    if request.method == 'POST':
        my_data = Data.query.get(request.form.get('id'))

        my_data.name = request.form['name']
        my_data.email = request.form['email']
```

```python
        my_data.phone = request.form['phone']

        db.session.commit()
        flash("Employee Updated Successfully")

        return redirect(url_for('Index'))




#This route is for deleting our employee
@app.route('/delete/<id>/', methods = ['GET', 'POST'])
def delete(id):
    my_data = Data.query.get(id)
    db.session.delete(my_data)
    db.session.commit()
    flash("Employee Deleted Successfully")

    return redirect(url_for('Index'))




if __name__ == "__main__":
    app.run(debug=True)
```

In this code we have created our Mysql Database configuration. you can see that our database name is crud.

```python
#SqlAlchemy Database Configuration With Mysql
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql://root:''@localhost/crud'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

And this is our database model.

```python
class Data(db.Model):
    id = db.Column(db.Integer, primary_key = True)
    name = db.Column(db.String(100))
    email = db.Column(db.String(100))
    phone = db.Column(db.String(100))


    def __init__(self, name, email, phone):

        self.name = name
        self.email = email
        self.phone = phone
```

This is our *base.html*.

templates/base.html

```html
<!DOCTYPE html>
<html lang="en">
<head>


    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap

    <meta charset="UTF-8">
    <title>{% block title %} {% endblock %} </title>
</head>
<body>

{% block body %} {% endblock %}


<script src="https://code.jquery.com/jquery-3.4.1.slim.min.js" integrity="sha
<script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.0/dist/umd/popper.mi
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/js/bootstrap.

</body>
</html>
```

In here we have created
our *header.html*.

templates/header.html

```
templates/header.html
1  {% extends 'base.html' %}
2
3
4
5  {% block title %} Flask CRUD {% endblock %}
6
7
8
9  {% block body %}
10
11 <div class="jumbotron p-3">
12     <div class="well text-center">
13
14         <h1>Python Flask CRUD Web Application Development </h1>
15
16     </div>
17
18
19 </div>
20
21
22 {% endblock %}
```

And this is our
index.html.

templates/index.html

```
templates/index.html
1  {% extends 'base.html' %}
2  {% include 'header.html' %}
3
4
5
6  {% block title %} Home {% endblock %}
7
8
9
10 {% block body %}
11
12 <div class="container">
13     <div class="row">
14         <div class="col md-12">
15
16             <div class="jumbotron p-3">
17
18                 <h2>Manage <b>Employees </b>  <button type="button"  class="
19                 data-toggle="modal" data-target="#mymodal">Add New Employees
20
```

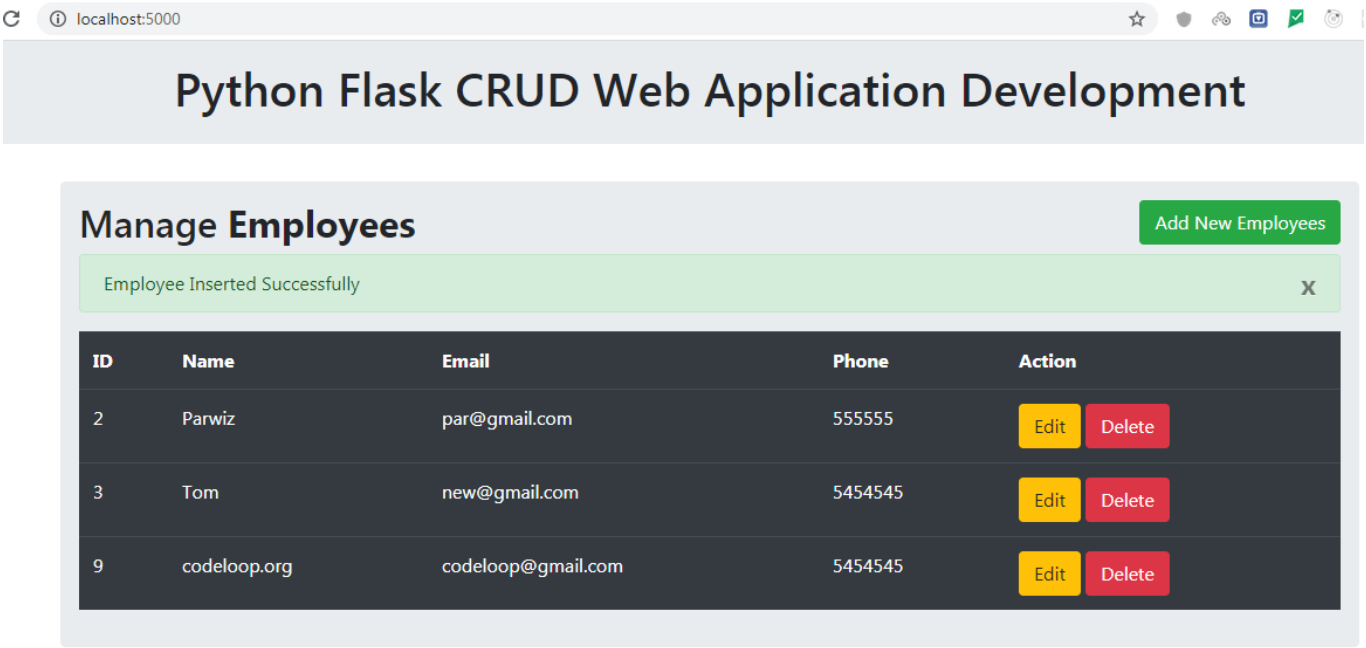
```
                    {% with messages = get_flashed_messages() %}

                    {% if messages %}

                    {% for message in messages %}

                    <div class="alert alert-success alert-dismissable" role="ale

                        <button type="button" class="close" data-dismiss="alert"

                            <span aria-hidden="true">x</span>

                        </button>


                    {{message}}


                    </div>


                    {% endfor %}

                    {% endif %}
                    {% endwith %}


                    <table class="table table-hover table-dark">

                        <tr>

                            <th>ID</th>
                            <th>Name</th>
                            <th>Email</th>
                            <th>Phone</th>
                            <th>Action</th>

                        </tr>


                       {% for row in employees %}
                        <tr>
                            <td>{{row.id}}</td>
                            <td>{{row.name}}</td>
                            <td>{{row.email}}</td>
                            <td>{{row.phone}}</td>

                            <td>
                                <a href="/update/{{row.id}}" class="btn btn-warn
                                <a href="/delete/{{row.id}}" class="btn btn-dang


                            </td>

                        </tr>


                      <!-- Modal Edit Employee-->
<div id="modaledit{{row.id}}" class="modal fade" role="dialog">
   <div class="modal-dialog">


       <div class="modal-content">
```

```
            <div class="modal-header">


               <h4 class="modal-title">Update Information</h4>


            </div>


            <div class="modal-body">


               <form action="{{url_for('update')}}" method="POST">


                  <div class="form-group">


                     <label>Name:</label>
                      <input type="hidden"  name="id" value="{{row.id}}">

                     <input type="text" class="form-control" name="name" value=


                  </div>



                  <div class="form-group">


                     <label>Email:</label>

                     <input type="text" class="form-control" name="email" value


                  </div>


                  <div class="form-group">


                     <label>Phone:</label>

                     <input type="text" class="form-control" name="phone" value


                  </div>


                  <div class="form-group">


                     <button class="btn btn-primary" type="submit">Update</butt


                  </div>


               </form>


            </div>
```

```
151
152          <div class="modal-footer">
153
154
155             <button type="button" class="btn btn-secondary" data-dismiss="mo
156
157
158          </div>
159
160
161       </div>
162
163
164    </div>
165
166 </div>
167
168
169                     {% endfor %}
170
171
172                 </table>
173
174
175             </div>
176
177
178
179                   <!-- Modal Add Employee-->
180
181
182 <div id="mymodal" class="modal fade" role="dialog">
183    <div class="modal-dialog">
184       <div class="modal-content">
185          <div class="modal-header">
186
187             <h4 class="modal-title">Add Employee</h4>
188          </div>
189          <div class="modal-body">
190
191             <form action="{{url_for('insert')}}" method="POST">
192
193
194                <div class="form-group">
195
196                   <label>Name:</label>
197                   <input type="text" class="form-control" name="name" requir
198
199
200                </div>
201
202
203                 <div class="form-group">
204
205
206                   <label>Email:</label>
207                   <input type="email" class="form-control" name="email" requ
208
209
210                </div>
211
212
213                 <div class="form-group">
214
215
```

```
                    <label>Phone:</label>
                    <input type="number" class="form-control" name="phone" req


                </div>



                <div class="form-group">


                    <button class="btn btn-primary" type="submit">Add Employee


                </div>


            </form>


        </div>


        <div class="modal-footer">


            <button type="button" class="btn btn-secondary" data-dismiss="mo


        </div>


      </div>


   </div>


</div>




        </div>

    </div>


</div>


{% endblock %}
```

So now run the complete project and this will be the result.



Flask CRUD Application with SQLAlchemy

Google Ads Philippines
Ad Google Ads

Request a Twilio Demo
Ad Twilio

Python Flask Create News Web Application Codeloop
codeloop.org

Expert MUMPS Developers (GTM - InterSystems) 25+ years...
Ad 4wtech.com

Flask Archives

Flask Archives
codeloop.org

How to Build REST API with SQLAlchemy & Marshmallow Codeloop
codeloop.org

Flask Tutorial - Flask SQLAlchemy with MySQL Codeloop
codeloop.org

Python IP Lookup Application with Flask Codeloop
codeloop.org

Django Apps & URL Routing Example Codeloop
codeloop.org

Flask Login &
Logout Tutor
codeloop.org

Flask Web D
Full Course f
Codeloop
codeloop.org

Ad
1. Crud Em Python
2. Sqlalchemy Course
3. Gerador De Crud PHP

Parwiz Forogh

YouTube

Like 540 people like this. Be the first of your friends

Flask, Python Tutorials

Flask, Flask CRUD, Flask SQLAlchemy, Python, SQLAlchemy

How to Create FontComboBox in Pyside2

Flask Web Development Tutorials

Python Flask Create News Web Application

Flask CRUD Application with SQLAlchemy

How to Build REST API with SQLAlchemy & Marshmallow

Flask Web Development Full Course for Beginners

Flask How to Create Routes with Flask-Classy

What is Flask – Python Web Framework

Python Flask – Introduction to Flask Templates

Flask Tutorial – Template Inheritance in Flask

Flask Tutorial – Template Variables in Flask

Flask Tutorial – Adding Bootstrap Styles to Flask

Flask Tutorial – Custom Error Page in Flask

Flask Tutorial –
Creating Dynamic
URL in Flask

Flask Tutorial –
Flask Forms with
Flask-WTF

Flask Tutorial –
Flask-WTF Form
Error Messages

Flask Tutorial –
Creating Flask Flash
Messages

Flask Tutorial –
Flask SQLAlchemy
with MySQL

Flask WTF
Registration Form
with SQLAlchemy

Flask Login & Flask
Logout Tutorial

How to Upload Files
With Python Flask

Python IP Lookup Application with Flask

Copyright © 2018 - 2020
Codeloop. All Rights Reserved